

## CampusEats Orders Assignment 5

### CampusEats Orders — Assignment 5: HTTP Methods & Headers

**Team ID: 24 Team members (Roll No · Name):** - 20251651068 · Paras Pandey - 20251651067 · Pankaj Singh - 20251651069 · Pranav Bhaoshar - 20251651049 · Jayesh Badole - 2025165182 · Shivam Phogat

This builds directly on the Assignment 4 Orders service (create / read / list / cancel). Assignment 5 adds OPTIONS discovery, conditional requests, content negotiation, auth, rate limiting, CORS and security headers on top of it.

#### A1 · CampusEats method map

| Method  | URL                                               | Action                                       | Success                             | Failure                      |
|---------|---------------------------------------------------|----------------------------------------------|-------------------------------------|------------------------------|
| OPTIONS | /orders                                           | Discover supported methods                   | 204                                 | —                            |
| POST    | /orders                                           | Place a new order                            | 201 (200 on Idempotency-Key replay) | 400, 401, 406, 422, 429, 503 |
| GET     | /orders?student={id}&status=&sort=&limit=&offset= | List/filter/sort/paginate a student's orders | 200                                 | 400, 401, 406                |
| OPTIONS | /orders/{id}                                      | Discover supported methods                   | 204                                 | —                            |
| GET     | /orders/{id}                                      | Read a single order                          | 200 (304 on If-None-Match hit)      | 401, 404, 406                |
| PATCH   | /orders/{id}                                      | Modify qty on a still-placed order           | 200                                 | 400, 401, 404, 409, 412      |
| POST    | /orders/{id} + X-HTTP-Method-Override: PATCH      | Fallback for clients that can't send PATCH   | 200                                 | 405                          |
| OPTIONS | /orders/{id}/cancel                               | Discover supported methods                   | 204                                 | —                            |
| POST    | /orders/{id}/cancel                               | Cancel an order                              | 202 (200 on Idempotency-Key replay) | 401, 404, 409                |

No verb leaked into a URL — every non-CRUD action ( `cancel` ) is `POST /orders/{id}/cancel` , never `POST /cancelOrder` .

We did **not** add a `DELETE /orders/{id}` . That is a deliberate, carried-over decision from Assignment 4: an order is a financial record and must not disappear from the store once payment has been captured. Its only valid state transition away from `placed` is `cancelled` , which is why cancellation is modelled as a sub-resource action instead of a destructive verb.

#### A3 · Safe / idempotent audit

| Endpoint         | Safe? | Idempotent? | Notes                                                   |
|------------------|-------|-------------|---------------------------------------------------------|
| GET /orders/{id} | Yes   | Yes         | Read-only; repeating it never changes state.            |
| GET /orders      | Yes   | Yes         | Read-only; query params only ever filter, never mutate. |
|                  |       |             |                                                         |

|                          |     |                                     |                                                                                                                                                                                                                                                                                                       |
|--------------------------|-----|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OPTIONS *                | Yes | Yes                                 | Pure discovery, no body, no state touched.                                                                                                                                                                                                                                                            |
| POST /orders             | No  | <b>Yes — by construction</b>        | Creates a resource, so <i>not</i> naturally idempotent. Made idempotent with a client-supplied Idempotency-Key : replaying the same key returns the original 201 result as a 200 , with no second payment attempt.                                                                                    |
| PATCH /orders/{id}       | No  | <b>No</b>                           | Repeating the identical request is refused the second time (its own If-Match value goes stale the instant the first call succeeds), so it is <i>not</i> idempotent in the strict sense — but it is retry-safe: the second identical attempt fails closed (412) instead of double-applying the change. |
| POST /orders/{id}/cancel | No  | <b>No — the one that is neither</b> | Cancelling an already-cancelled order fails with 409 , not the same 202 result.                                                                                                                                                                                                                       |

**How we made cancel retry-safe:** the same Idempotency-Key mechanism as create — an optional Idempotency-Key header on POST /orders/{id}/cancel . First call: 202 + the key is remembered against that order. A client retry with the *same* key (e.g. because it never saw the response due to a dropped connection) gets back the original 200 result instead of a 409 . A **second, distinct** caller cancelling again without a key still correctly gets 409 — that’s a genuine state conflict, not a retry.

Confirmed no GET ever changes state (both read handlers only call store.find / store.findByStudent , never store.create or any mutator).

## C4 · Safe-retry plan

| Endpoint                 | Risk if retried blindly                                                                                  | Mechanism              | Why this one                                                                                                                                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST /orders             | Double order, double charge                                                                              | <b>Idempotency-Key</b> | The risk is a duplicate <i>create</i> , not a stale read of an existing resource — a key that survives across retries is the only mechanism that can recognize “this is the same attempt” before the resource even exists.          |
| PATCH /orders/{id}       | Lost update — two editors silently overwrite each other’s change                                         | <b>If-Match</b>        | The resource already exists and already has a state (ETag); the danger is clobbering a change made by someone else since the client last read it, which is exactly what a precondition check on the current representation catches. |
| GET /orders/{id}         | Not a mutation risk, but wasted bandwidth on an unchanged resource                                       | <b>If-None-Match</b>   | Purely a freshness question on a safe read — the client already knows the version it has; confirming “still the same?” needs nothing stronger than a comparison.                                                                    |
| POST /orders/{id}/cancel | Same double-submission risk as create (e.g. a client retries a timed-out cancel and gets a spurious 409) | <b>Idempotency-Key</b> | Cancel is also a create-like “do this exactly once” action on top of a mutable resource — the id already exists, but the <i>action</i> still needs a way to be                                                                      |

recognized as “I already tried this,” which points to the same key mechanism as create rather than a precondition header.

This follows directly from A3: the safe × idempotent split tells you which mechanism applies — **safe reads get If-None-Match**, **unsafe writes to an existing representation get If-Match**, and **unsafe actions that are naturally non-idempotent (create-like) get an Idempotency-Key**.

## D2 • Headers table

| Endpoint                                              | Request headers it needs                                                                                     | Response headers it sets                                                                                                                |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| OPTIONS /orders, /orders/{id}, /orders/{id}/cancel    | — (no auth required for discovery)                                                                           | Allow, CORS headers                                                                                                                     |
| POST /orders                                          | Content-Type: application/json, Authorization: Bearer <token>, Idempotency-Key (optional), Accept (optional) | Location, ETag, X-RateLimit-Limit, X-RateLimit-Remaining, Content-Type, CORS headers, X-Content-Type-Options, Strict-Transport-Security |
| GET /orders?student=                                  | Authorization: Bearer <token>, Accept (optional)                                                             | Cache-Control: no-store, X-RateLimit-*, Content-Type, CORS + security headers                                                           |
| GET /orders/{id}                                      | Authorization: Bearer <token>, If-None-Match (optional), Accept (optional)                                   | ETag, Cache-Control: private, max-age=30, must-revalidate, X-RateLimit-*, CORS + security headers                                       |
| PATCH /orders/{id} (or POST + X-HTTP-Method-Override) | Content-Type: application/json, Authorization: Bearer <token>, If-Match ( <b>required</b> )                  | ETag (new value), X-RateLimit-*, CORS + security headers                                                                                |
| POST /orders/{id}/cancel                              | Authorization: Bearer <token>, Idempotency-Key (optional)                                                    | X-RateLimit-*, CORS + security headers                                                                                                  |
| Every response, always                                | —                                                                                                            | Date (added automatically by Node’s HTTP server)                                                                                        |
| Every request, always (rate-limited routes)           | —                                                                                                            | X-RateLimit-Limit, X-RateLimit-Remaining; Retry-After added only on 429                                                                 |
| Any request from a browser on another origin          | Origin (sent by the browser)                                                                                 | Access-Control-Allow-Origin, Access-Control-Allow-Methods, Access-Control-Allow-Headers, Access-Control-Expose-Headers                  |

## Eight answers

### 1. Three endpoints — method, success status, the one response header that matters most, and why

- POST /orders → **201** → **Location** — the client’s very next action (read, patch, cancel) needs the new order’s URL, and 201 without it is nearly useless.
- GET /orders/{id} → **200** → **ETag** — every conditional-request mechanism in this service (304 on GET, 412 on PATCH) is anchored on this one value; without it, C1 and C2 have nothing to compare against.
- PATCH /orders/{id} → **200** → **ETag** (the *new* value) — the client must have it to make its next edit; reusing the old ETag would make its very next PATCH fail with a 412 against its own successful write.

### 2. Which endpoints are safe, which are idempotent, which is neither, and how was it made retry-safe?

Safe and idempotent: GET /orders/{id}, GET /orders, all OPTIONS routes. POST /orders is unsafe but

made idempotent via `Idempotency-Key`. `PATCH /orders/{id}` is unsafe and not strictly idempotent (its own `If-Match` value expires the instant it succeeds), but it is retry-safe because a replayed request fails closed with 412 instead of double-applying. **`POST /orders/{id}/cancel` is the one that is neither safe nor idempotent** — cancelling an already-cancelled order returns 409, not the original 202. We made it retry-safe the same way as create: an optional `Idempotency-Key` header, so a client retrying its own dropped request gets the original result back instead of a spurious conflict, while a genuinely separate second cancel attempt still correctly gets 409.

### 3. One ETag, the 304, and the 412 — what does each save/prevent?

From the captured transcript: `GET /orders/2` returned `ETag: "5fb425a239d0f2a26ae027cd4a25e66d32eea39e"`. Repeating that GET with `If-None-Match: "5fb425a239d0f2a26ae027cd4a25e66d32eea39e"` returned **304 Not Modified** with no body — this saves the ~139 bytes of JSON (and the work of re-serializing it) every time a client re-checks an order it already has cached. Sending `PATCH /orders/2` with a stale `If-Match: "stale-etag-value"` (the resource had already moved to the `5fb42...` ETag) returned **412 Precondition Failed** and left `qty` untouched — this prevents a lost update: the client was about to write on top of a version of the order it hadn't actually seen.

### 4. 422 for one case, 400 for another — exact requests and the difference

**400**: `POST /orders` with `{"studentId":"not_an_int"}` — the request body doesn't even parse into a valid `OrderRequest` (wrong types, missing required fields). This is a **syntax/schema** problem caught by `validateCreate()` before any business logic runs; the server never attempted to act on the request.

**422**: a structurally valid `POST /orders` body ( `studentId`, `itemId`, `qty`, `paymentMethodId` all present and correctly typed) whose `paymentMethodId` the downstream payments service declines — `paymentsClient` catches that as a `CardDeclined` error and the handler returns **422 payment-declined**. This is a **semantic** refusal: the request was well-formed, was understood, and was acted on, but the specific business action (charging that card) could not be completed.

### 5. A cross-origin browser call is blocked, but the server logs a 200 — who blocked it, and what fixes it?

The **browser** blocked it, not our server. The server did complete the request and did return 200 — that's exactly why it's in the logs — but the browser's own same-origin policy withholds the response from the page's JavaScript unless the response carries an `Access-Control-Allow-Origin` header naming (or wildcarding) the calling origin. We fixed it in `cors.js` by setting `Access-Control-Allow-Origin: *` (plus `Access-Control-Allow-Methods / -Headers`) on every response and by answering the CORS preflight `OPTIONS` request the browser sends before the real one.

### 6. One response cached, one no-store — why each?

**Cacheable**: `GET /orders/{id}` → `Cache-Control: private, max-age=30, must-revalidate`. A single order is read far more often than it changes, it belongs to one student (hence `private`, not a shared/CDN cache), and pairing it with the `ETag` means a stale copy is *revalidated* (cheap 304) rather than trusted blindly past 30 seconds.

**no-store**: `GET /orders` (the list). A filtered/paginated collection changes on every new order or cancellation, is parameterized per-request ( `status`, `sort`, `limit`, `offset` ), and caching it risks either serving a stale list or — worse — a shared cache returning one student's filtered view to a different caller. It's cheap enough to just always re-run the query.

### 7. The search/list endpoint is a GET — when would POST be right instead, and what do you give up?

`GET /orders` would need to become (or be joined by) a `POST /orders/search` once the filter criteria get too large or too sensitive to sit in a URL — e.g. searching across a long list of item IDs, a free-text query, or anything a client shouldn't have sitting in browser history, a shared link, or a proxy/server access log in plain sight. A JSON body has no practical length limit and doesn't get logged the way a query string does. What you give up by switching: safety and idempotence in the HTTP sense (a `POST` is not something a browser will cache, prefetch, or safely auto-retry), bookmarkability/shareability (you can't copy-paste a

POST as a link), and the free conditional-GET machinery ( ETag / If-None-Match ) this assignment relies on for the list.

### 8. Location on a 201 vs. on a 3xx — what does it point to in each case?

On **201 Created** ( POST /orders ), Location points to the resource that request just created — /orders/{id} for the order that now exists because of this call. On a **3xx redirect**, Location points to a *different* resource the client should now request *instead* of the one it asked for — it carries no implication that anything was created; it just says “what you want lives over here now.”

---

### D1 · curl -v transcript

---

See curl-transcript.txt for the full transcript (run with bash capture-transcript.sh against the live service + the Assignment 4 dummy bank). It captures, in order: a successful create ( 201 + Location + ETag ), the same create repeated with the same Idempotency-Key ( 200 , original order, no second charge), a conditional GET returning 304 , a conditional PATCH rejected with 412 and a second PATCH with the correct If-Match succeeding, a 400 (malformed create body), a 404 (unknown order), a 401 (missing bearer token), an OPTIONS discovery call, and a cancel followed by a second cancel showing 202 then 409 .

### Tests

---

npm test — 19/19 passing ( jest-output.txt ), covering auth, content negotiation, OPTIONS/Allow, conditional GET/PATCH, the create *and* cancel idempotency-key paths, rate limiting, CORS preflight, and security headers.